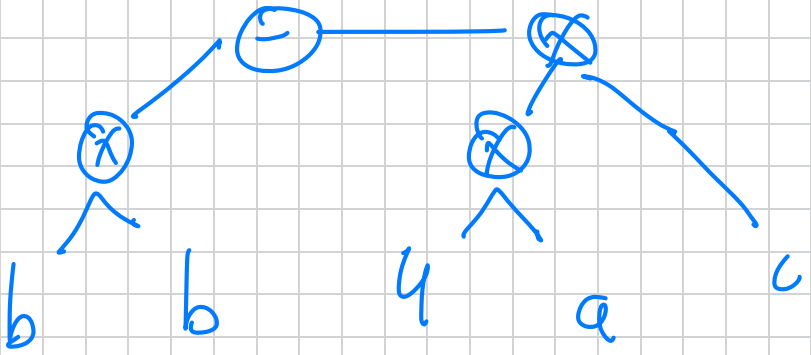
TD 1 :

1. Machine à pile :

$\Delta = b^2 - 4ac$, entier 16 bits complément à 2.

1. arbre d'évaluation :



2. 3. Programme :

calcule de b^2

```
push B
push B
mul
```

calcule de $4ac$

```
push 4
push a
mul
push c
mul
```

calcule de $b^2 - 4ac$

```
sub
```

pile

4 -

1	push	b
2	push	b
3	mul	
4	push	4
5	push	a
6	mul	
7	push	c
8	mul	
9	sub	

Résultat pile :

b
b, b
b^2
4
4, a
4a
4a, c
4ac
$b^2 - 4ac$

2. Machine à accumulateur :

1. nombre de case mémoires :

			Resultat Pile :
1	Push	b	b
2	Push	b	b, b
3	mul		b^2
4	Push	4	4
5	Push	a	4, a
6	mul		4a
7	Push	c	4a, c
8	mul		4ac
9	Sub		$b^2 - 4ac$

b, a, c

b^2 , 4ac , $b^2 - 4ac$

Donc 6 cases mémoires.

2. Pseudo - code :

```
Load    mem [b]
Mul     mem [b]
Store   mem [b²]

Load    cst (4)
Mul     mem (a)
Mul     mem (c)
Store   mem (4ac)

Load    mem (b²)
Sub     mem (4ac)
Store   mem (Delta)
```

TD 2.

1. Branchements conditionnels :

1.1 introduction :

		R17	R16	Z	N	C
LDI	R16, 10	✓	10	✓	✓	✓
CPI	R16, 9	✓	10	0	0	0
BANE	test					
SUB	R16, 2					
LSL	R16					
test :						
LDI	R17, 6	6	10	✓	✓	✓
ADD	R16, R17	0	16	0	0	0
(décalage) LSL	R16	6	32	0	0	0
SUBI	R16, 1	6	31	0	0	0

R16 : 0000 1100

R17 : 0000 0110

R16 : 0001 0000 / 0010 0000

1.2 Structure de contrôle :

algorithme :

```
R18 ← A
Si (R18 <= 10) alors
    R20 ← 12
Sinon
    R25 ← 30
fin si
```

```
Ldi   R18, 0
Cpi   R18, 10
BRGE  X :
Ldi R25, 30
Jump fin si
X: Ldi R20, 12
fin si.
```

instruction assembleur atmega 2560 :

pour : (R18 ≤ 10) : Cpi R18, 10 — ça n'existe pas. — ~~BRLE~~ X

(R18 ≥ 10) : Cpi R18, 10 ; BRGE X

(R18 < 10) : cpi R18, 10 ; BRLT X

(R18 > 10) : Cpi R18, 10 ; BRGT X

```
Ldi R18, 0
Cpi R18, 10
bRGE test
Ldi R25, 30
Jump fin si
test : Ldi R20, 12
fin si
```

```
Ldi R16, 12
Ldi R17, 5
Cpi R16, 0
BRLT fin tantque
MV R22, R16
add R22, R17
dec R16
Jump tantque
fin tantque
```

2.

R16 ← 12
tant que (R16 >= 0) faire
    R22 ← R16 + S
    R16 ← R16 - 1
fin tant que.

```
Ldi     R16, 12
Ldi     R17, S
ETQ:    tst R16 ?
BRGE    loop.
JMP end_loop.
loop:   mov     R22, R16
        add     R22, R17
        dec     R16
        Jump    ETQ
end_loop.
```

R16 ← 12
(R16 >= 0)
R16 = 0 ? tst R16

R17 ← S
R22 ← R16
R22 ← R22 + S
R16 ← R16 - 1
retour en début de boucle
fin tant que.

3.

max ← t[0]
Pour tout n de 1 jusqu'a nmax
  si max < t[n] alors
    max ← t[n]
  fin si
fin pour.

```
ldi R15, 1          <- 8 bits
ld  R16, lo8(t1):xL
ld  R17, hi8(t1), xH
ld  R17, xt valeur max
boucle: cpi R15, Nmax
        breq finalgorithme
        cp R16, xt
        brlt suite
suite:  mov R17, R16
        inc R15
        Jump Boucle
fin tant que.
```

Pour xL x Low
         x High

2. Mode adressage :

a : @ 00

2.1 : Exercice 1 :

a => 16 bits @ $0100

b => a (+1 5)

@ Direct

c

a = 0B F2

R17 R16

@101 @101

R16 ← mem [100]

R17 ← mem [101]

R18 ← 15$_{10}$ ← col

R17, R16 ← R17, R16 + R18

mem [102] ← R16

mem [103] ← R17

@ Indirect :

R28, R29

pfb pfr

0X 01 00

R29 R28

R28 ← 00

R29 ← 01

R16 ← mem [Y]

R28 ← R28 + 1

R17 ← mem [Y]

R17, R16 ← R17, R16 + 15

R28 ← R28 + 1

mem [Y] ← R16

R28 ← R28 + 1

mem [Y] ← R17

ldi R18, 15

ldi R29, 00 XL

ldi R29, 01 XH

ld R16, Y

INC R28

ld R17, Y

Adiw R16, R18

INC R28

ST Y, R16

INC R28

st Y, R17

@ Indirect avec dép const :

R28 ← 00

R29 ← 01

R16 ← mem [Y]

R17 ← mem [Y+1]

R17, R16 ← R17, R16 + 15

mem [Y+2] ← R16

mem [Y+3] ← R17

ldi R18, 15

ldi R29, 00 XL

ldi R29, 01 XH

ld R16, Y

ld R17, Y + 1

Adiw R16, R18

ST Y+2, R16

st Y+3, R17

@ indirect avec post inc.

R28 ← 00
R29 ← 01

R16 ← mem [Y+]
R17 ← mem [Y+]

R17, R16 ← R17, R16 + 15
mem [Y+3] ← R16
mem [Y+] ← R17

ldi R18, 15
ldi R29, 00 XL
ldi R29, 01 XH

ld R16, Y+
ld R17, Y+

adiw R16, R18
st Y+, R16
st Y+, R17